



MASTER IN ASTROPHYSICS
AND SPACE SCIENCE



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Optimisation & Monte Carlo Sampling

INTRODUCTION TO NUMERICAL METHODS FOR ASTROPHYSICS

Fritz Ali Agildere

November 27, 2025

Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Frequentist View

- *With Prior Knowledge*: out of all n different but equally likely possible outcomes of an event, there are k that have a property A which is therefore measured with probability

$$P(A) = \frac{k}{n}$$

- *Without Prior Knowledge*: out of n independent measurements the outcome A occurs k times, defining

$$P(A) = \lim_{n \rightarrow \infty} \frac{k}{n}$$

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B
- *Kolmogorov Axioms* underpin the rules for calculating with probabilities
- $P(A, B) = P(A) P(B|A)$ quantifies the probability of measuring A and B simultaneously, and from assuming $P(A, B) = P(B, A)$ we find

Bayes' Theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Continuous Distributions

■ *Cumulative Distribution Function:* $F(x) = P(X \leq x)$

■ *Probability Density Function:* $f(x) = \frac{d}{dx}F(x)$

$$P(a \leq X \leq b) = \int_a^b f(x) dx = F(b) - F(a)$$

■ *Normalization:* $1 = \int_{-\infty}^{+\infty} f(x) dx$ $\lim_{x \rightarrow -\infty} F(x) = 0$ $\lim_{x \rightarrow +\infty} F(x) = 1$

Expectation Values

$$E[h(x)] = \int_{-\infty}^{+\infty} h(x) f(x) dx$$

■ *Mean* or first raw moment: $\mu = E[X] = \int_{-\infty}^{+\infty} x f(x) dx$

■ *Variance* or second raw moment: $\sigma^2 = E[(X - E[X])^2] = \int_{-\infty}^{+\infty} (x - E[x])^2 f(x) dx$

Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Drawing Numbers

$$x \sim U(a, b)$$

$$y \sim N(\mu, \sigma)$$

■ *Uniform Distribution:* $U(a, b) = f(x|a, b) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$

■ *Gaussian Distribution:* $N(\mu, \sigma) = f(y|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \left(\frac{y-\mu}{\sigma}\right)^2\right)$

Number Generation

- *Pseudo Random Numbers: $x_{j+1} = g(x_1, \dots, x_j)$*
 - deterministic & reproducible
 - efficient & transparent
 - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
 - *Linear Congruential*
 - *Linear Feedback Shift*
 - *Permutated Congruential*
- *Arbitrary Distributions:* obtained from uniformly distributed numbers by various methods

Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Motivation

- *Model Optimisation*: finding the best possible model parameters
 - maximising objective function (*Fitting*)
 - minimising loss function (*Machine Learning*)
- *Example*: modified blackbody spectrum for galactic dust emission (M. Juvela, A&A 2023)

$$I_{\nu}(T, \beta) \equiv \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

Example

$$I_{\nu}(T, \beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*
 - independent measurements generated by $I_{\nu}(\theta)$ with normally distributed uncertainty
 - infinitesimally small error in ν so that we only need σ_{ν} in I_{ν} direction
 - suppose we have no prior information on the parameter distribution

Example

- *Likelihood:*

$$-\log L(I_v | v, \sigma_v, \theta) \propto \chi^2(\theta) = \sum_{i=1}^N \frac{(I_{v,i} - I_v(\theta))^2}{\sigma_{v,i}^2}$$

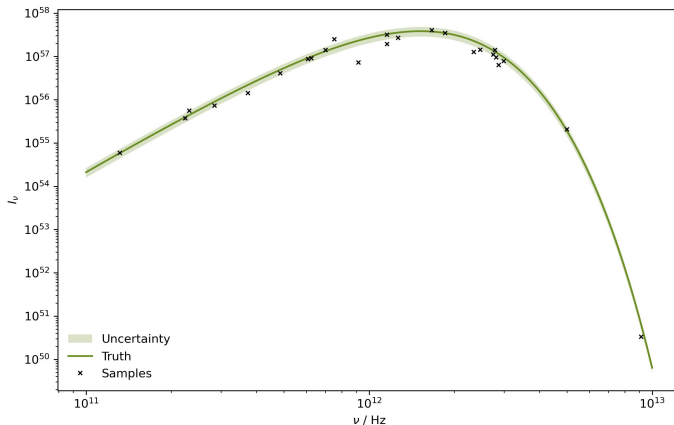
- *Posterior:*

$$\log P(\theta | I_v, v, \sigma_v) \propto -\chi^2(\theta)$$

- *Objective:* minimise the $\chi^2(\theta)$ measure to maximise the posterior probability

Measurements

How do we proceed from this dataset to maximise the likelihood for our nonlinear model?



Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

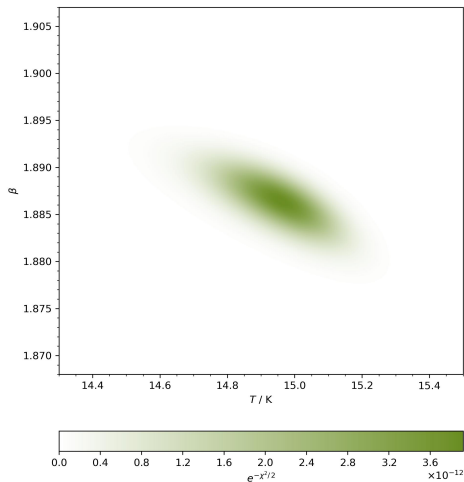
Alternatives

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter
 - from the literature we expect $(T, \beta) \in [10, 25] \times [1.5, 2.0]$ as constraints
 - resize these intervals by defining $T \mapsto \tilde{T} = T / 30$ with $I_v(T, \beta) \rightarrow I_v(\tilde{T}, \beta)$
- *Problem:* as D increases we have N^D gridpoints, and with more complex parameter distributions, this procedure quickly becomes intractable due to computational limitations

Parameter Grid

Can we think of more elegant ways to explore even very high dimensional parameter spaces?



General Case

- *Optimisation Algorithms*: intelligently move through parameter space towards optimum
- *Common Attributes*:
 1. choose initialisation point
 2. evaluate objective function or its derivatives to estimate local curvature
 3. propose new step in parameter space
 4. decide whether proposal is a good step and update position accordingly
 5. repeat until some predefined cutoff is satisfied
- *Interactive Visualisations*: check out Ben Frederickson¹ and Andy Casey² to see some nice examples

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
 1. initialize simplex of $d + 1$ points in d dimensional parameter space
 2. rank points in order of values $f(x_1) \leq \dots \leq f(x_d) \leq f(x_{d+1})$
 3. compute centroid x_0 of $x_1, \dots, x_d$
 4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
 5. reflect, expand, contract or shrink based on results to move points downhill and repeat
- *Notes:*
 - extrapolates function to slowly move geometric blob through parameter space
 - zeroth order method since no derivatives are used

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
 1. start from arbitrary initial value x_0
 2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
 3. propose updated position $\hat{x}_{k+1} = x_k + X$
 4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*
 - traverses parameter space via modified random walk, can be very slow
 - zeroth order method since no derivatives are used
 - improve by considering last step and optimizing step range

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
 1. initialize at some x_0
 2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
 3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
 - moves along direction of steepest descent at each point with fixed learning rate α
 - first order method since only the gradient is used
 - improve by performing line search for α at every step

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
 1. start from some fixed large α_0 for each step k of the gradient descent
 2. set $\alpha_{j+1} = \tau \alpha_j$ until $f(x_k) - f(x_k - \alpha_j \nabla f(x_k)) \geq \kappa \alpha_j (\nabla f(x_k))^2$ where $\tau, \kappa \in (0, 1)$
 3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
 - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily decreases with increasing k
 - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$
 - robust convergence guarantees, larger step size for flat areas avoids saddle points
 - first order method since only the gradient is used
 - typical settings are $\tau = \kappa = 0.5$ (L. Armijo, *Pac. J. Math.* 1966)

Conjugate Gradient Descent

■ *Requirements:* first derivatives of the objective function, can be estimated numerically

■ *Algorithm:*

1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \frac{\nabla f(x_k)(\nabla f(x_k) - \nabla f(x_{k-1}))}{(\nabla f(x_{k-1}))^2}$
2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached

■ *Notes:*

- uses gradients from previous evaluations to estimate curvature
- smooths oscillatory behaviour of naive gradient descent, especially in tight valleys with nearby gradients almost orthogonal to search direction
- first order method since only the gradient is used

Adaptive Moment Estimation

■ *Requirements*: first derivatives of the objective function, can be estimated numerically

■ *Algorithm*:

1. initialize position $\mathbf{x}_0$ as well as moving averages $\mathbf{m}_0 = \mathbf{0}$ and $\mathbf{v}_0 = \mathbf{0}$
2. set $\mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1 - \beta_1) \nabla f(\mathbf{x}_k)$ and $\mathbf{v}_{k+1} = \beta_2 \mathbf{v}_k + (1 - \beta_2) (\nabla f(\mathbf{x}_k))^2$
3. correct estimator bias with $\tilde{\mathbf{m}}_k = \mathbf{m}_{k+1} (1 - \beta_1^k)^{-1}$ and $\tilde{\mathbf{v}}_k = \mathbf{v}_{k+1} (1 - \beta_2^k)^{-1}$
4. update the position $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \tilde{\mathbf{m}}_k (\sqrt{\tilde{\mathbf{v}}_k} + \epsilon)^{-1}$ and repeat

■ *Notes*:

- point behaves as iterative analogue to ball rolling in landscape with friction
- hyperparameters $\beta_1, \beta_2 \in (0, 1)$ are forgetting factors for previous gradients, and $\epsilon \ll 1$
- first order method since only the gradient is used

Newton Method

■ *Requirements:* first and second derivatives of the objective function, can be estimated numerically

■ *Algorithm:*

1. choose some initial x_0

2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i} \right)_{i=1,\dots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j} \right)_{i,j=1,\dots,d}$

3. update the parameters $x_{k+1} = x_k - \alpha H_{f(x_k)}^{-1} \nabla f(x_k)$ with dampening α

■ *Notes:*

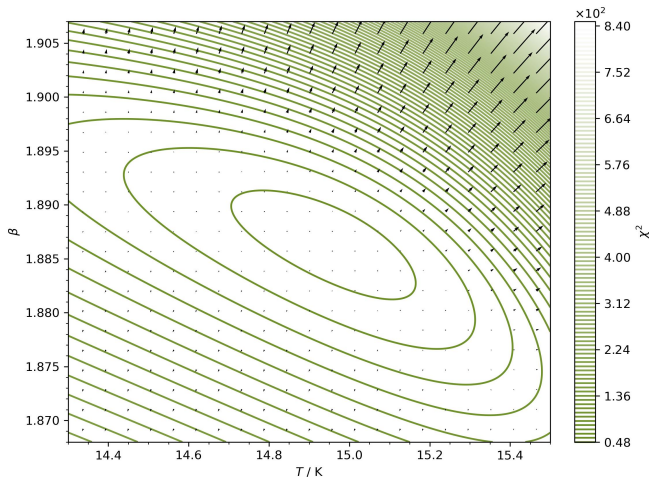
- generalises Newton-Raphson method for finding roots of derivative to multiple dimensions
- finds critical point, no matter if maximum, minimum or saddle point, sensitive to initialisation
- second order method since both gradient and Hessian are used
- related Gauss-Newton algorithm employs Jacobian instead of Hessian

Advanced Algorithms

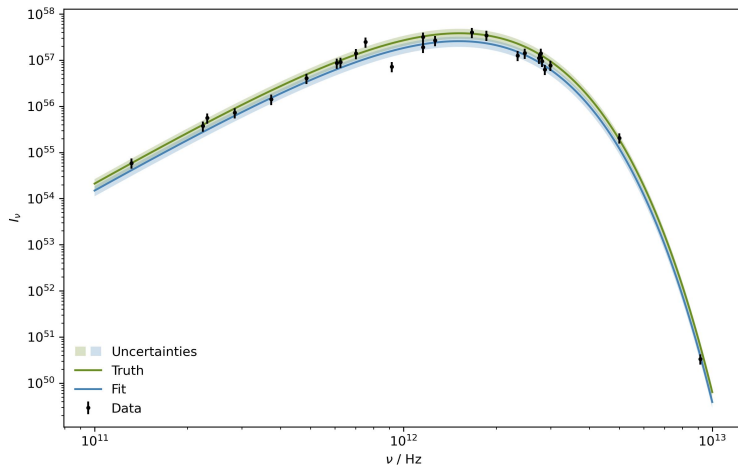
- *Traits*: quasi second order methods, interpolate between optimisers, various curvature estimations
 - Broyden–Fletcher–Goldfarb–Shanno
 - Sequential Least Squares Programming
 - Levenberg–Marquardt
 - Migrad
 - Root Mean Square Propagation
- *Usage*: found in many modern libraries, suited to specialised use cases like nonlinear least squares, problems with parameter bounds or arbitrary constraints

Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$
 - $T_{400} = 14.96 \text{ K}$ & $\beta_{400} = 1.866$
- *Line Search:* $\alpha_0 = 10^{-4}$ & $\tau = \kappa = 0.5$
 - $T_{40} = 14.96 \text{ K}$ & $\beta_{40} = 1.866$



Result



Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Limitations & Curses

■ *Convergence & Silent Failures:*

- algorithm continues until convergence threshold is met or optimisation fails
- check output for which termination reason applies
- verify that parameters have actually changed from initial guess

■ *Dimensionality:*

- volume and distance become somewhat meaningless in higher dimensions
- impossible to determine where to step or which parameter is important
- gridding uniformly in linear or logarithmic space is expensive but a useful option
- fixing some parameters or optimising independently are potential solutions

Limitations & Curses

- *Scaling:*

- parameter scalings of different magnitudes produce pathological stopping criteria

- *Nonconvexity:*

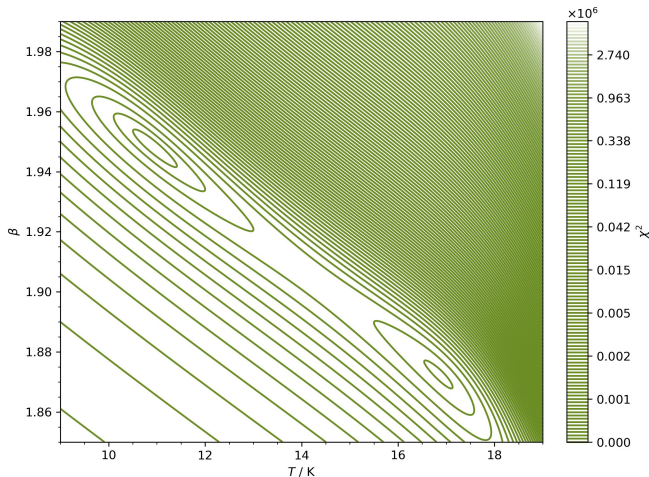
- convergence does not guarantee that the global optimum has been found
 - basin hopping or simulated annealing as potential fixes

- *Initialisation:*

- optimisation can converge to different local optima depending on starting point and algorithm

Limitations & Curses

- Example: $T_0 = 13.8 \text{ K}$ & $\beta_0 = 1.8615$
 - Fixed Rate Gradient Descent:
 $T_{2700} = 16.86 \text{ K}$ & $\beta_{2700} = 1.872$
 - Line Search Gradient Descent:
 $T_{660} = 10.93 \text{ K}$ & $\beta_{660} = 1.948$



Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Motivation

- *Optimisation*: finds a point estimate of optimal model parameters
- *Monte Carlo Sampling*: dynamically explores the parameter space distribution
 - quote asymmetric percentiles
 - estimate expectation values by integrating
 - uncertainty propagation to the model function
 - simulate complex measurement processes and quantify the total convolution

Markov Chain

■ *Properties:*

- sequence generated by stochastic transitions
- transition probabilities depend only on the initial and final state

■ *Detailed Balance:*

- any chain belonging to a Markov Process represents a reversible sequence of events
- when walking along a stationary distribution $f(\mathbf{x})$ the following must hold for every transition probability $M_{i \rightarrow j}$ from state $\mathbf{x}_i$ to $\mathbf{x}_j$

$$M_{i \rightarrow j} f(\mathbf{x}_i) = M_{j \rightarrow i} f(\mathbf{x}_j)$$

- then the average behaviour of this single process reproduces that over the entire ensemble

Monte Carlo

■ *Randomness:*

- each proposed next step is accepted or discarded based on a probabilistic condition
- if the underlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated
- else it is based on a random choice with acceptance being less likely for smaller $f(x_{k+1})$

■ *Rules:*

- the proposal distribution is allowed to violate detailed balance only if it is corrected in the decision between acceptance and rejection
- unlike the underlying posterior that we sample, the proposal must be properly normalized

Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Markov Chain Monte Carlo

■ Metropolis:

- a proposal x_j is drawn from a fixed step size distribution such as a normal or uniform one that is symmetric in each coordinate $g(x_j|x_i) = g(x_i|x_j)$
- this satisfies detailed balance $M_{i \rightarrow j} f(x_i) = M_{j \rightarrow i} f(x_j)$ with

$$M_{i \rightarrow j} = \min \left(1, \frac{f(x_j)}{f(x_i)} \right)$$

- generate a random number $\xi \sim U(0, 1)$ and decide according to

$$x_{i+1} = \begin{cases} x_j & \xi \leq M_{i \rightarrow j} \\ x_i & \xi > M_{i \rightarrow j} \end{cases}$$

Markov Chain Monte Carlo

■ *Metropolis-Hastings:*

- a proposal x_j is drawn from an asymmetrix distribution $g(x_j|x_i) \neq g(x_i|x_j)$
- this itself violates detailed balance but is fixed by $M_{i \rightarrow j} f(x_i) g(x_j|x_i) = M_{j \rightarrow i} f(x_j) g(x_i|x_j)$ with

$$M_{i \rightarrow j} = \min \left(1, \frac{f(x_j) g(x_i|x_j)}{f(x_i) g(x_j|x_i)} \right)$$

- generate a random number $\xi \sim U(0, 1)$ and decide according to

$$x_{i+1} = \begin{cases} x_j & \xi \leq M_{i \rightarrow j} \\ x_i & \xi > M_{i \rightarrow j} \end{cases}$$

Markov Chain Monte Carlo

■ *Comparison:*

- the Metropolis-Hastings is more flexible than the Metropolis algorithm since it can use asymmetric proposal distributions
- asymmetric proposals allow efficient handling of bounded parameter spaces
- proposal distribution shapes can be tailored to the sampled posterior to speed up exploration of the space and achieve good convergence
- realistic cases, especially when priors are included, are generally not symmetric or unbounded, therefore benefitting from the inbuilt bias correction to ensure correct sampling of the target distribution

Advanced Methods

■ *Affine-Invariant:*

- multiple walkers that spread out into parameter space controlled by stretch parameter
- biased proposals are made along a line connecting randomly paired walkers and corrected in the decision step
- very efficient at sampling correlated parameters due to reduced hyperparameters
- suitable in low dimensional parameter spaces where no gradient information is known

Advanced Methods

■ *Adjusted Langevin:*

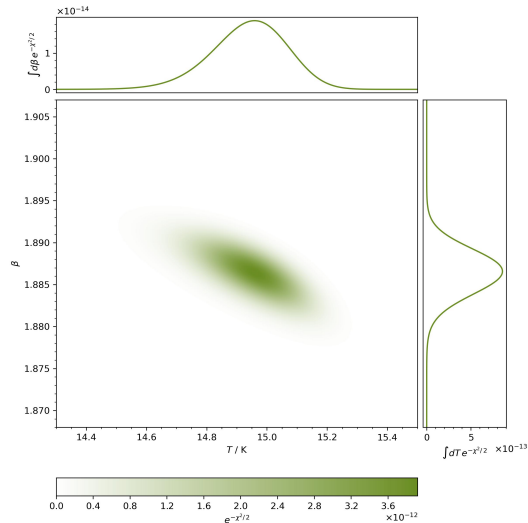
- uses gradient of logarithmic likelihood or score function as a drift force pushing samples towards higher probability
- a small Gaussian noise term is added to act as a random force to allow exploration of the distribution
- bias is corrected with a Metropolis-Hastings decision step
- application in gradient estimation and diffusion learning

Advanced Methods

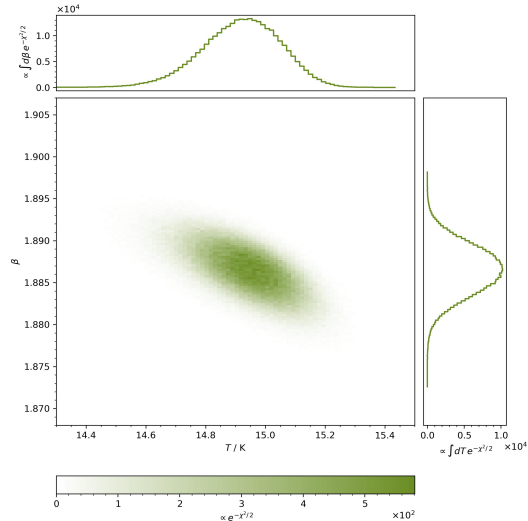
■ *Hamilton:*

- treat target function as potential energy, introduce random momentum term for kinetic energy
- definition in canonical distribution ensures separability so that $\mathbf{x}$ and $\mathbf{p}$ are independent
- for each step, accurately integrate momentum some set amount of steps by using the gradient, conditioned by the Hamiltonian or total energy remaining constant
- check proposal with Metropolis decision metric to account for numerical integration errors
- permits very long directed jumps that drastically reduce the correlation of the chain
- efficient for many dimensions with very high acceptance ratios

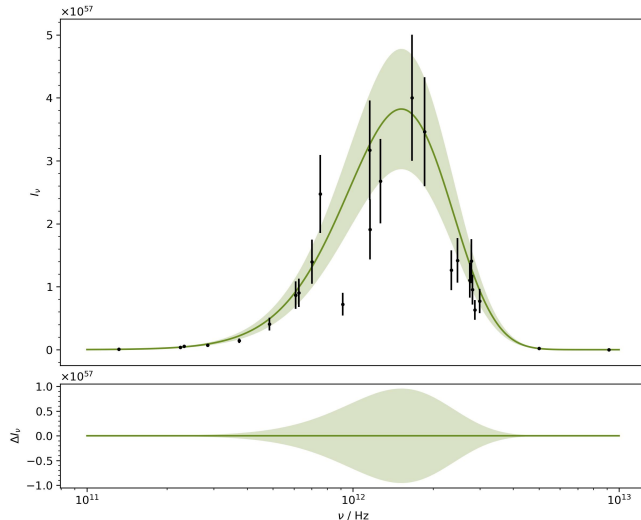
Posterior



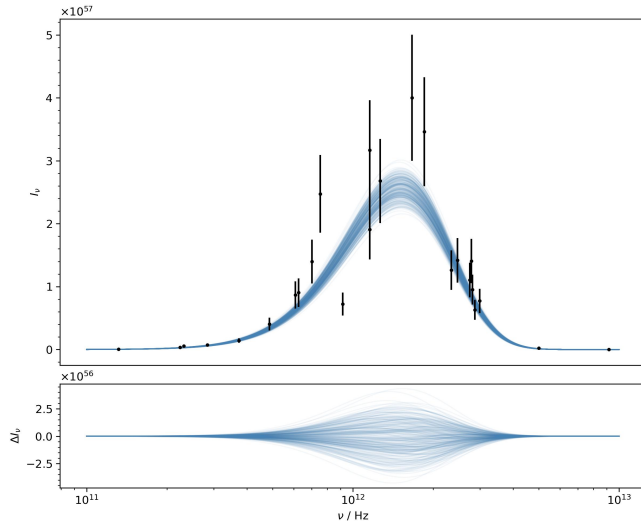
Posterior



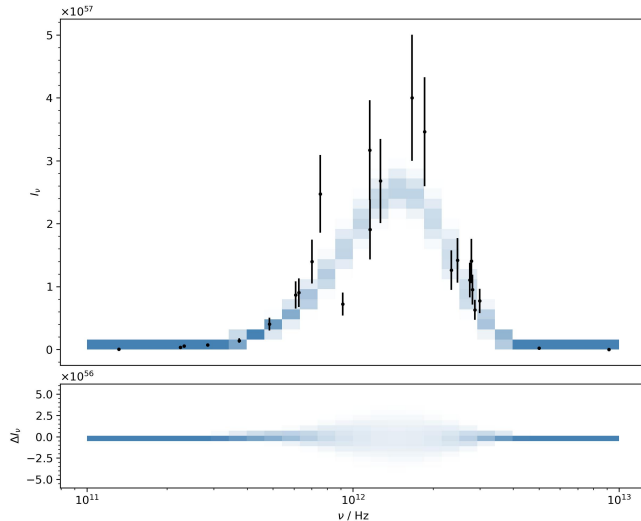
Comparison



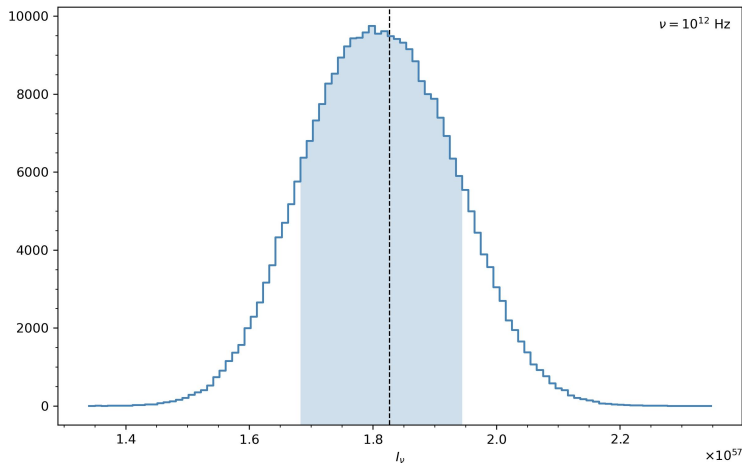
Comparison



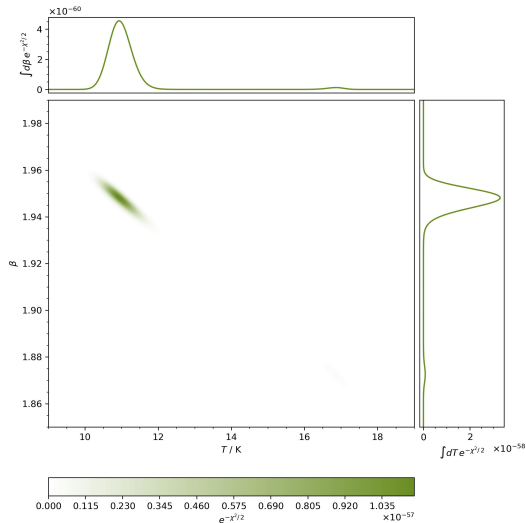
Comparison



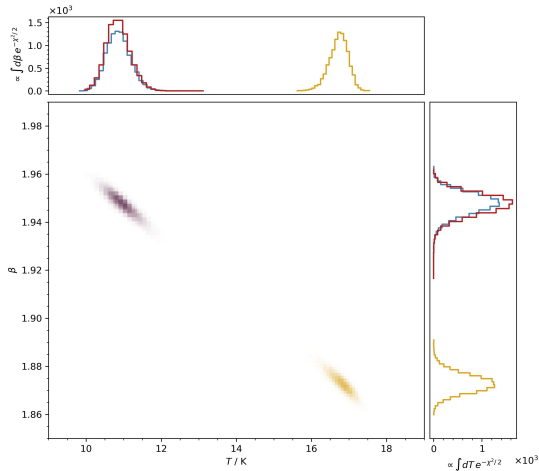
Error Propagation



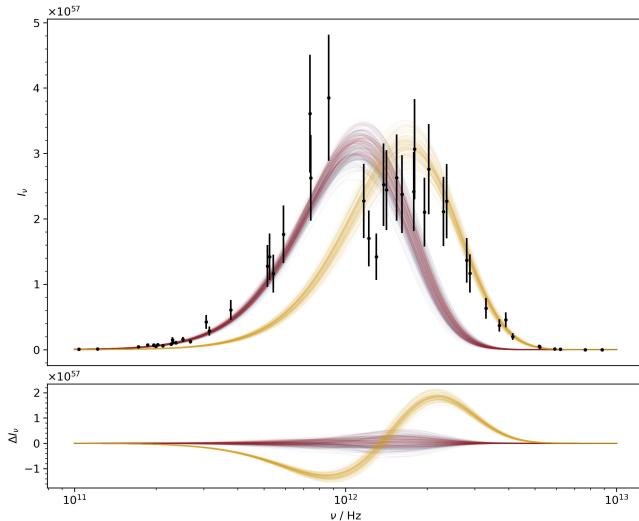
Posterior



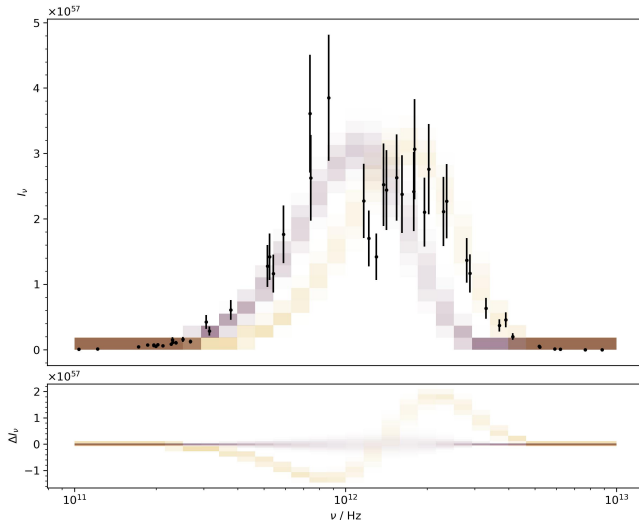
Posterior



Comparison



Comparison



Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Sampling Efficiency

- *Low Acceptance Rate: $\lesssim 30\%$*
 - only small fraction of significant part of target explored
 - few points actually represent the distribution
 - step size too large
- *High Acceptance Rate: $\gtrsim 70\%$*
 - many points close to each other
 - flat part of target distribution
 - missing large portions of relevant parameter space
 - step size too small

Burn In

■ *Problem:*

- initialisation far from significant part of target
- flat distribution, majority of points not relevant
- no convergence to important portions of parameter space

■ *Fixes:*

- use ensemble of walkers
- identify burn in phase via trace plot and reject it

Autocorrelation

■ Definition: $\rho_t = \frac{\text{Cov}(X_k, X_{k+t})}{\sqrt{\text{Var}(X_k)\text{Var}(X_{k+t})}} = \frac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$

- Pearson correlation coefficient $\rho_t \in [-1, 1]$ for one parameter with a lagged version of itself
- Markov Chain elements are correlated with their neighbours since they are generated from their current states
- effective sample size is reduced by autocorrelation scale
- compute expectation values for N samples by summing only to $M \ll N$ to avoid signal being dominated by statistical noise and after discarding burn in

Autocorrelation

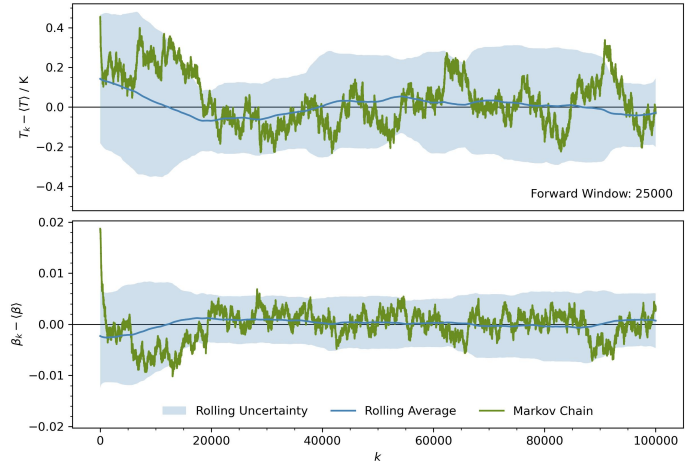
- Confidence: $\sigma_t^2 = \frac{1}{M} \left(1 + 2 \sum_{s=1}^t \rho_s^2 \right)$

- Bartlett estimate for uncertainty σ_t of ρ_t by a truly random process
- autocorrelation length l is defined by lag t after which $|\rho_t| < \sigma_t$ and therefore $\rho_t \neq 0$ no longer statistically significant

Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

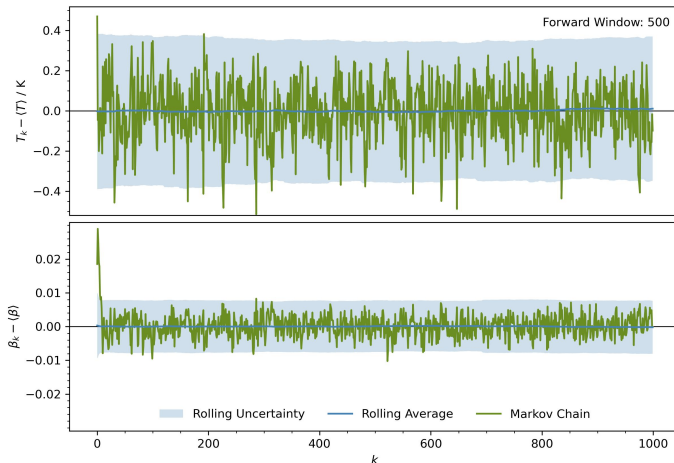
- $h = 10^{-4}$ & acc = 97.72%
- acceptance too high
- step size too short



Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

- $h = 10^{-2}$ & acc = 12.13%
- acceptance too low
- step size too long

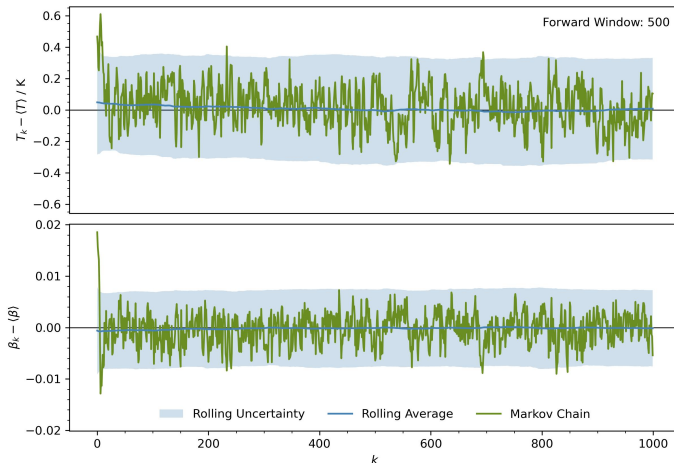


Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

■ $h = 5 \cdot 10^{-3}$ & acc = 30.27%

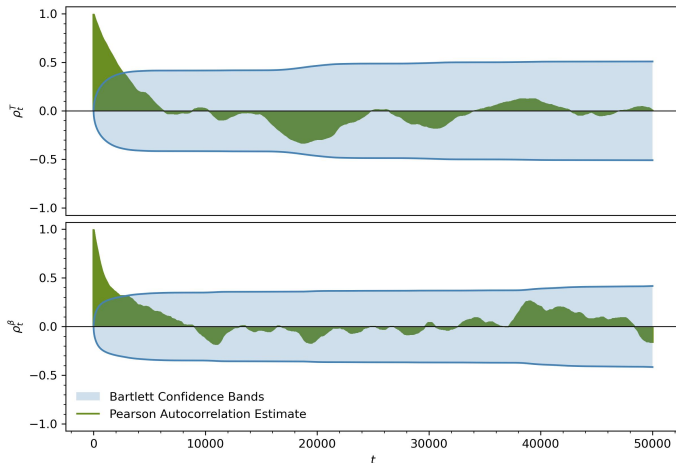
■ acceptance and step size
seem good



Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

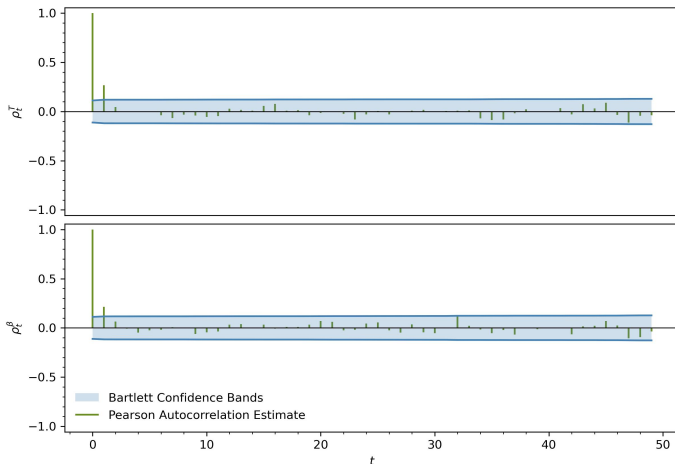
- $h = 10^{-4}$ & acc = 97.72%
- acceptance too high
- step size too short
- $l \approx 3000$



Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

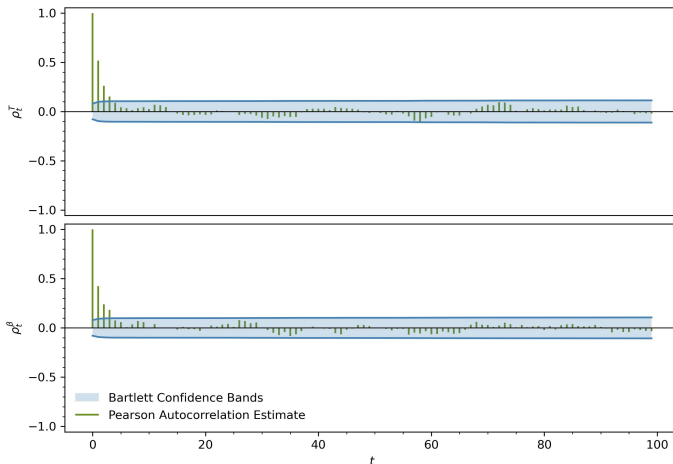
- $h = 10^{-2}$ & acc = 12.13%
- acceptance too low
- step size too long
- $l = 2$



Application

■ Metropolis Algorithm: $x_j \sim N(x_i, h)$

- $h = 5 \cdot 10^{-3}$ & acc = 30.27%
- acceptance and step size seem good
- $l = 4$



Overview

Probability Refresher

Definitions & Notation

Random Numbers

Optimisation

Common Methods

Problems

Markov Chain Monte Carlo

Popular Algorithms

Tuning & Diagnosing

Alternatives

Shortcomings

- *Expensive*: should only be used when absolutely necessary, do not do the following
 - sample the entire parameter space
 - improve poor optimisation performance
 - discard distributions at intermediate steps
- *Other Tools*:
 - *Inversion Sampling*: transform uniform random numbers to desired distribution
 - *Rejection Sampling*: propose samples, then accept or reject based on density
 - *Importance Sampling*: weight samples from simple density to match target

Thank You!

- A. Casey, *Data Analysis and Machine Learning*³ 2022
- M. Linhoff & W. Rhode, *Statistical Methods of Data Analysis* 2023
- *emcee documentation*⁴ 2025

Resources on GitHub: [here](#)